Taller de Proyecto II

Informe Final

F.5. Comparación de Receptores de Radio FM HEX3653 y TEA5767

Grupo de Desarrollo

- Madera, Ramiro - 02956/2
- Pereira, Ulises - 02981/3

1.- Descripción General del Proyecto

El objetivo inicial del proyecto es la implementación de un sistema de control para los receptores de radio **HEX3653** y **TEA5767**, para una posterior comparación en el desempeño entre ellos. En ambos casos, esto implica el diseño esquemático de las conexiones a la placa principal de trabajo, que inicialmente era una **ESP32**, y la programación de una interfaz web con HTML plano que permitiera ejecutar las funciones de control de cada receptor. Para el módulo HEX, la comunicación por interfaz web implica la programación de pulsadores digitales reemplazarían el uso de botones físicos en el dispositivo; para el módulo TEA, implica la programación mediante comunicación I2C con el mismo.

Conforme se desarrolló el trabajo, se realizaron ciertas modificaciones a la idea inicial. En primer lugar, se decidió fusionar los 2 sistemas independientes pensados para cada receptor en un sistema integrado que permita manejar ambos en paralelo; la alimentación del sistema, originalmente pensada para realizarse a través de una fuente externa por conexión USB, fue reemplazada por la utilización de dos pilas AA que permitieran la portabilidad del sistema; finalmente, se decidió cambiar la placa de desarrollo a una placa **ESP8266**, debido a su adaptabilidad a los requerimientos del proyecto y al número de pines de alimentación y GND que la misma posee.

A este proyecto inicial se le incorporaron 2 extensiones, ambas relacionadas a la amplificación de la salida de sonido; esto, debido a una falta de pines en la placa, fue realizado solo para el receptor TEA, dejando de lado el trabajo con el HEX para esta segunda parte. En primera instancia, se diseñó un sistema de alimentación que pueda regular el volumen con un potenciómetro físico; y luego, se diseñó un sistema que permita manejarlo con un potenciómetro digital cuyo control sería agregado a la interfaz web.

2.- Presentación Esquemática del Proyecto Inicial

2.1.- Gráfico de procesos y funciones

El funcionamiento del sistema se se resume en dos procesos principales:

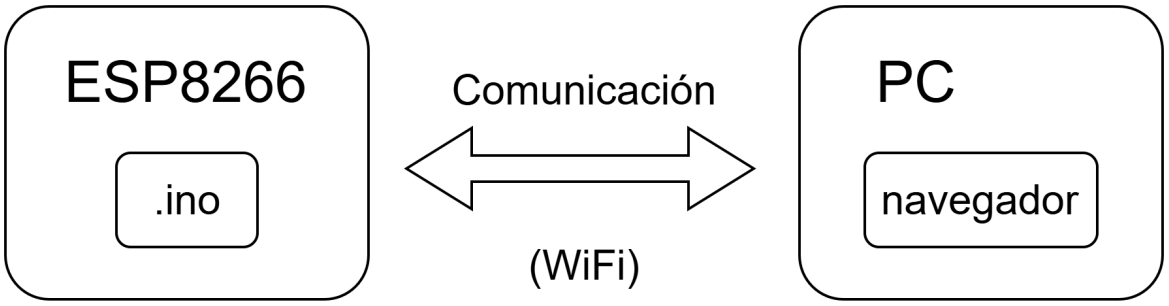

Figura 1: Diagrama de procesos del sistema.

El proceso con el cual interactúa directamente el usuario se ejecuta en el navegador de una PC (u otro dispositivo inteligente con capacidad de navegar por una red WiFi), este es el lugar desde donde se envían requerimientos para controlar los módulos FM. Luego, el otro proceso se ejecuta en el ESP8266; es el que levanta la red a la cual se conecta la PC, implementa la interfaz web, maneja los requerimientos generados por ésta y ejecuta todas las funciones que en última instancia manejan los módulos. La comunicación entre ambos procesos se da por la red WiFi generada por el ESP8266.

A continuación se exponen los esquemas de las interfaces web, junto con las funciones que debe satisfacer el sistema en cada caso:

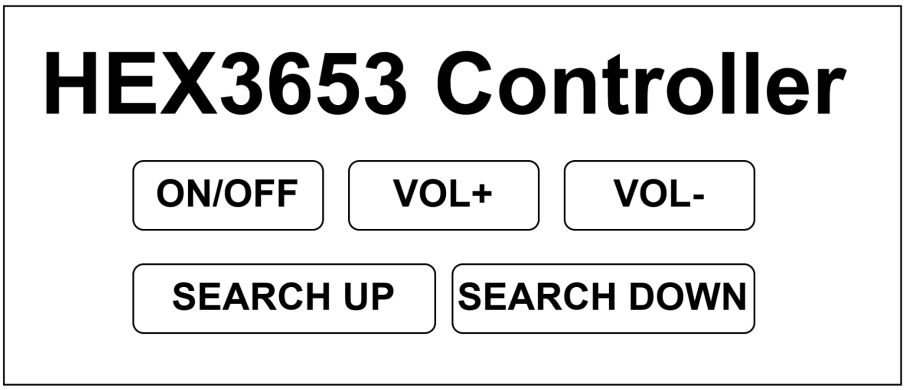

Figura 2: Esquema de la interfaz web del HEX3653.

El controlador del HEX3653 posee 5 botones que guardan una correlación directa con los switches físicos que se reemplazaron en el módulo original:

- **ON/OFF:** Enciende y apaga el HEX3653.
- **VOL+ / VOL-:** Incrementan y decrementan el volumen de la salida de audio.
- **SEARCH UP / SEARCH DOWN:** Sintonizan la próxima estación con una señal estable (que no sea solo ruido) buscando desde la estación actual hacia arriba (UP) o hacia abajo (down).

Figura 3: Esquema de la interfaz web del TEA5767.

El controlador del TEA5767 tiene 5 botones, un campo para ingresar números y dos secciones que otorgan información se la estación sintonizada:

- **STAND-BY:** Ingresa y saca al módulo del modo “stand-by”.
- **MUTE:** Mutea o desmutea la salida de audio.
- **TUNE FQ:** Sintoniza la frecuencia ingresada en el campo adyacente.
- **SEARCH UP / SEARCH DOWN:** Funciona de la misma manera que el caso anterior.
- **CURRENTLY TUNNING:** Muestra la frecuencia de la estación que está siendo sintonizada en ese momento.
- **WITH QUALITY:** Muestra el número de calidad que le otorga el TEA5767 a la estación sintonizada en ese momento.

Es de interés aclarar, que si bien en una primera instancia se plantearon dos interfaces separadas, durante el desarrollo del proyecto se tomó la decisión de implementar el manejo simultáneo de los dos módulos, por lo cual la interfaz final incluye ambos controladores con todas las funciones descritas.

Se considera redundante detallar en particular los casos de uso específicos del sistema, debido a que la interfaz web es transparente respecto a la funcionalidad que cumple cada botón, y los mismos funcionan de forma independiente entre sí.

2.2.- Esquemático de conexiones

En la siguiente figura se adjunta el esquemático completo del proyecto, con el controlador conectado a ambos módulos FM y siendo alimentado con un pack de dos baterías AA, en un holder que originalmente pertenecía al kit del HEX3653.

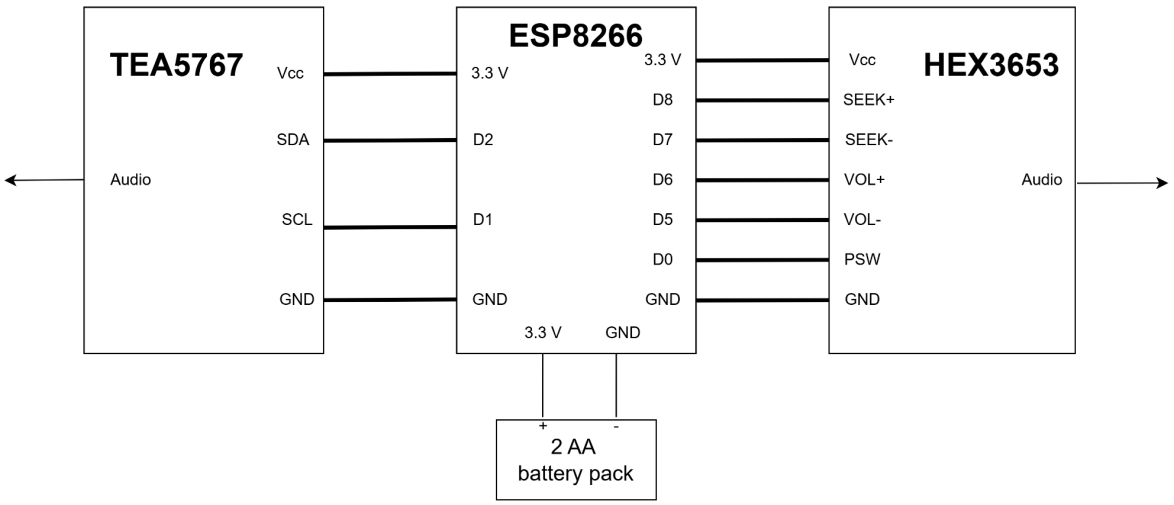

Figura 4: Diagrama de conexión del proyecto original.

La alimentación de ambos módulos, así como la del ESP8266 con las baterías, fue posible porque se contó con un encapsulado de éste último que poseía un grán número de pines de 3.3V y GND para alimentar varios periféricos; esta cualidad es específica de la placa elegida y es lo que permite realizar el esquemático previo. De tratarse de otra placa, se debería haber buscado una alternativa para poder proveer de alimentación a todos los componentes conectados al ESP.

2.3.- Alimentación del proyecto

En la siguiente tabla se muestran los requerimientos energéticos del sistema:

Tabla 1: Requerimientos energéticos del proyecto original.

Componente	**Consumo máximo (mA)**	**Tensión de alimentación (V)**
ESP8266	170	2.5 - 3.6
TEA5767	15.5	2.5 - 5
HEX3653*	30	3

* No se ha encontrado una hoja de datos que detalle el consumo del HEX3653, pero éste se puede inferir por comparación con integrados similares empleados en radios portátiles de bajo costo (como el KT0936), que suelen consumir entre 15 y 30 mA. El módulo se alimenta con dos pilas AA, por lo que requiere una tensión de alrededor de 3V.

Esto lleva a un total de **215.5 mA**, cantidad que un par de pilas alcalinas AA, que comúnmente cuentan con una capacidad de 2400mAh, pueden alimentar por aproximadamente 11 horas.

2.4.- Cumplimiento de requerimientos iniciales

A continuación se enumeran los **requerimientos primarios** del plan de proyecto original:

HEX3653

1. Reemplazo “digital” de pulsadores para la generación de pulso de las funciones.
2. Provisión de interfaz web con ESP8266 utilizando HTML plano, sin bibliotecas externas.
3. Propuesta de interfaz mejorada y análisis de costo/beneficio.
4. Análisis y evaluación del sistema completo.

TEA5767

1. Programación de las funciones y obtención de reportes de sintonización del módulo.
2. Definición heurística de “calidad”.
3. Provisión de interfaz web con ESP8266 utilizando HTML plano, sin bibliotecas externas.
4. Propuesta de interfaz mejorada y análisis de costo/beneficio.
5. Análisis y evaluación del sistema completo.

De los cuales fueron completados en su totalidad los que implican el control y comunicación de ambos módulos, llegando incluso al control secundario de los mismos, como también la implementación de la interfaz web en HTML plano y el análisis final del sistema completo.

En cuanto al punto de definición heurística de “calidad”, se cumplió de manera parcial, pues se logró recibir el reporte de calidad de señal proporcionado por el TEA5767 pero no se profundizó en este tema. El otro punto que no se realizó en su totalidad fue el de la propuesta de interfaz mejorada; se contempló con el docente la posibilidad de optimizar la comunicación del PC con el ESP8266 para mejorar la latencia en la interfaz de manejo, pero quedó como opción de extensión a futuro sin redactar ninguna propuesta formal.

Estos puntos eran parte de los **requerimientos secundarios** listados del plan de proyecto original:

1. Implementación de las propuestas mejoradas de las interfaces.
2. Utilización de un amplificador para las señales de audio.
3. “Eliminación” de frecuencias sintonizadas consideradas de baja calidad en el TEA5767.

De los cuales, luego de haber logrado el funcionamiento correcto de ambos módulos, se optó por desarrollar el de la amplificación de la señal de audio, proceso que será detallado en las siguientes secciones.

3.- Documentación del Software del Proyecto Inicial

El proyecto fue desarrollado con la placa ESP8266 en el IDE de Arduino, versión 2.3.6. Para poder trabajar sobre la placa se instaló el manejador de placa “esp8266” versión 3.1.2, dentro del cual se seleccionó la placa “Generic ESP8266 Module”. En adición a estos drivers, se utilizó otro controlador para poder trabajar con el receptor TEA5767, el cual fue tomado desde la plataforma Github y se adjunta en la documentación relacionada. Este código, sin embargo, no fue instalado o agregadas al IDE de Arduino, sino que fueron agregadas particularmente al proyecto como archivos de código.

A continuación, se realizará una descripción exhaustiva, archivo a archivo, del código, en el cual se mencionarán y detallarán tanto el objetivo como el trabajo de cada una de las funciones en ellos. Se terminará describiendo el archivo que contiene el código fuente de la interfaz web, en el cual se explicará cómo monta la interfaz y cómo debe utilizarse la misma.

1. *ProyectoF5Final*: En este archivo .ino se ejecuta el sistema en sí; tiene un *loop* que recibe las requests del punto Wifi, y un setup en el cual se inicializan las conexiones de ambos receptores y se levanta la interfaz HTTP;
2. *HTTPGeneral*: Estos son los archivos *.cpp* y *.h* en los cuales se encuentra todo el código que permite levantar la interfaz del sistema. El mismo contiene los siguientes elementos y funciones:
   - 2.1. El elemento servidor de la placa ESP8266, las direcciones IP que se utilizarán y el nombre y contraseña de la red Wifi que levantará la aplicación;
   - 2.2. La función *handleRoot()* que hace que el servidor asocie la ruta base con la página web;
   - 2.3. La función *HTTP_init()* en la cual se configura el Access Point, se asocia la ruta *“/”* con la página web, y se llama a la inicialización de las rutas relacionadas a ambos receptores FM;
   - 2.4. La función *WifiHTTP_handleReq()*, la cual se llama desde el *loop* inicial y permite manejar las requests de la página;
3. *HandlerHEX3653*: En estos archivos *.cpp, .h* se encuentra todo el código relacionado al manejo del receptor HEX3653. En los mismos:
   - 3.1. Se define en el *.h* las macros para asociar los pines del ESP8266 a los pines del receptor;
   - 3.2. Se define la función *pulso(int, unsigned int)*, que manda un pulso en valor bajo por un tiempo enviado por parámetro a un pin enviado por parámetro;
   - 3.3. Se definen los Handlers que se llamarán cuando se accionen los botones de la interfaz web:
     - *3.3.1.* *handleSUp()*: envía un pulso al pin *S_UP*, que busca la siguiente frecuencia superior a la actual, y notifica a la interfaz de ello;
     - 3.3.2. *handleSDown()*: envía un pulso al pin *S_DOWN*, que busca la siguiente frecuencia inferior a la actual, y notifica a la interfaz de ello;
     - 3.3.3. *handleVUp()*: envía un pulso al pin *V_UP*, que sube el volumen, y notifica a la interfaz de ello;
     - 3.3.4. *handleVDown()*: envía un pulso al pin *V_DOWN*, que baja el volumen, y notifica a la interfaz de ello;
     - 3.3.5. *handleOnOff()*: envía un pulso al pin *ON_OFF*, que invierte el estado de alimentación del receptor (prendido/apagado), y notifica a la interfaz de ello;

3.4. Se define la función *hexpins_init()*, la cual configura todos los pines del receptor como salidas, escribe un valor en alto en ellos, y llama a *handleOnOff()* para que se inicialice apagado;

3.5. Se define la función *hexHTTP_init()*, que asocia los handlers mencionados previamente con las rutas asociadas a cada botón en el código HTML;

4. *TEA 5767 y tea5767_core*: Estos archivos, tanto los *.cpp* como los *.h*, fueron tomados de una librería pública; su implementación no se considera relevante para los fines de este proyecto. Sí se debe rescatar que se define al controlador del receptor como una clase con los siguientes procedimientos:

4.1. El constructor de clase, con parámetros que se han dejado nulos;

4.2. *awake()*, que despierta al receptor;

4.3. *standby()*, que duerme al receptor;

4.4. *getInfo(info)* que devuelve la información del receptor en la clase mandada por parámetro. Los valores que interesan en este proyecto son: estado de despierto o dormido, muteado o desmuteado, la frecuencia que se encuentra recibiendo, y la calidad de la señal;

4.5. *mute()*, que mutea el receptor;

4.6. *unmute()*, que desmutea el receptor;

4.7. *searchUp()*, que busca la siguiente frecuencia superior de calidad relevante. Al utilizarse, se debe dejar un delay de aproximadamente 80 milisegundos para que el receptor funcione correctamente;

4.8. *searchDown()*, que busca la siguiente frecuencia inferior de calidad relevante. Al utilizarse, se debe dejar un delay de aproximadamente 80 milisegundos para que el receptor funcione correctamente;

4.9. *setFrequency(float)*, que setea la frecuencia al valor enviado por parámetro. Al utilizarse, se debe dejar un delay de aproximadamente 80 milisegundos para que el receptor funcione correctamente;

5. *HandlerTEA5767*: En estos archivos *.cpp, .h* se encuentra todo el código relacionado al manejo del receptor TEA5767. En los mismos se encuentra:

5.1. Una variable interna que recupera el estado del receptor, con sus múltiples variables;

5.2. Se define la función *updateHTML()*, que envía al servidor un mensaje con todos los parámetros del receptor que deben mostrarse en la interfaz (estado de suspensión, frecuencia actual, estado de muteo, calidad de señal);

5.3. Se definen los Handlers que se llamarán cuando se accionen los botones de la interfaz web:

5.3.1. *handleUp()*: Hace que el receptor busque la siguiente frecuencia superior, recupera el estado total y llama a *updateHTML()* para enviarlo a la interfaz;

5.3.2. *handleDown()*: Hace que el receptor busque la siguiente frecuencia inferior, recupera el estado total y llama a *updateHTML()* para enviarlo a la interfaz;

5.3.3. *handleMute()*: Hace que el receptor invierta el estado del sonido entre muteado y sonando, recupera el estado total y llama a *updateHTML()* para enviarlo a la interfaz;

5.3.4. *handleSleep()*: Hace que el receptor invierta su estado entre despierto y dormido, recupera el estado total y llama a *updateHTML()* para enviarlo a la interfaz;

5.3.5. *handleFreq(float)*: Hace que el receptor fije la frecuencia enviada por parámetro, recupera el estado total y llama a *updateHTML()* para enviarlo a la interfaz;

5.3.6.

5.4. La función *teafunc_init()*, la cual inicializa el receptor, lo desmutea, lo saca del estado de suspensión, lo setea a una frecuencia inicial de 92.1, y recupera la información del receptor;

5.5. La función *teaHTTP_init()*, que asocia los handlers mencionados previamente con las rutas asociadas a cada botón en el código HTML;

6. *InterfazWeb.h*: En este archivo *.h* se encuentra el código que ejecuta la interfaz web del sistema. El mismo es un código HTML que contiene los siguientes elementos y/o funciones:

6.1. Declaraciones de componentes estéticos tales como los botones para armar la interfaz del sistema;

6.2. Un encabezado a modo de “firma”, con el título del proyecto, autores y profesor a cargo;

6.3. Un grupo *Control HEX3653* con los siguientes 5 botones:

6.3.1. *S_UP*: Envía un pulso para buscar la siguiente frecuencia superior;

6.3.2. *S_DOWN*: Envía un pulso para buscar la siguiente frecuencia inferior;

6.3.3. *V_UP*: Envía un pulso para aumentar el volumen;

6.3.4. *V_DOWN*: Envía un pulso para bajar el volumen;

6.3.5. *ON/OFF*: Envía un pulso para cambiar el estado del HEX a prendido o apagado, según en lo que esté previamente, invirtiendo el estado;

6.4. Un grupo *Control TEA5767* con los siguientes botones:

6.4.1. *Seek* ↑, que busca la siguiente frecuencia superior de calidad considerable;

6.4.2. *Seek* ↓, que busca la siguiente frecuencia inferior de calidad considerable;

6.4.3. *Mute*, que silencia o des-silencia al receptor;

6.4.4. *Standby ON/OFF*, que duerme o despierta al receptor;

6.4.5. *Enviar Valor*, el cual también contiene un campo llamado *valorEntero*, el cual al ser activado manda el comando de setear la frecuencia al valor ingresado en el campo;

6.5. Un apartado debajo de *Control TEA5767* donde se muestra si el receptor está o no en *standby*, y de no estarlo, muestra la frecuencia actual, si está o no silenciado, y la calidad de señal;

6.6. La función *renderTEA(state)*, que recibe el estado por parámetro y se encarga de mostrarlo en la interfaz;

6.7. La función *cargarEstadoTEA()*, que busca el estado del receptor TEA y lo envía a la función *renderTEA()*;

6.8. Un listener que actualiza el estado del TEA llamando a la función *cargarEstadoTEA()* cada medio segundo;

6.9. La función *enviarPulso(pin)*, la cual activa la ruta enviada por parámetro y carga el estado del TEA. Esta función es invocada por los botones del HEX3653 y los botones del TEA5767 sistema que no envían valores numéricos;

6.10. La función *enviarValor()*, que activa la ruta de seteo de frecuencia específica para el TEA5767, rescatando el valor que se encuentra en el campo numérico que contiene. Esta función solo es llamada por ese botón;

Se omite la descripción detallada de casos de usos debido a la trivialidad de los mismos mencionada previamente; cada botón de la interfaz web, y cada función asociada a los mismos, son autoexplicativas del trabajo que realizan.

4.- Otra Documentación Relacionada

- En la Figura 5, se encuentra una foto del sistema de hardware completamente conectado y listo para funcionar.

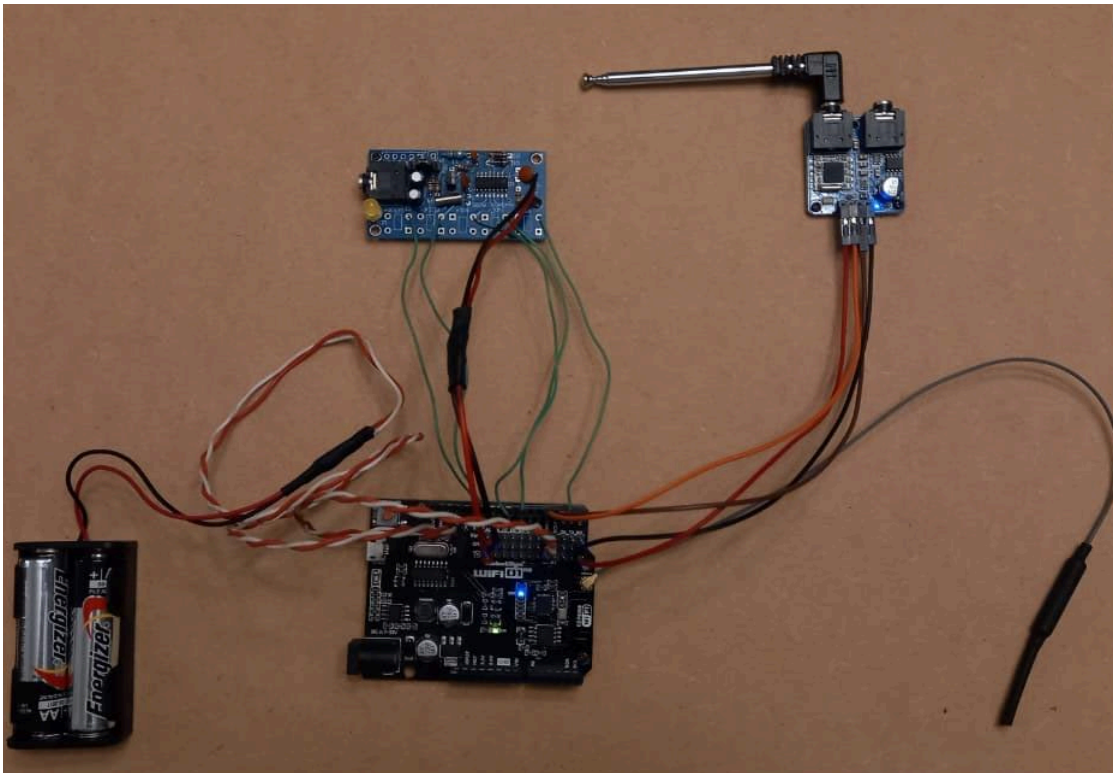

Figura 5: Hardware del proyecto completo

- En la Figura 6, se encuentra la foto del ESP8266 utilizado, evidenciando los múltiples puertos de GND y VCC que contiene, los cuales permitieron realizar la alimentación de la forma en la que se hizo.

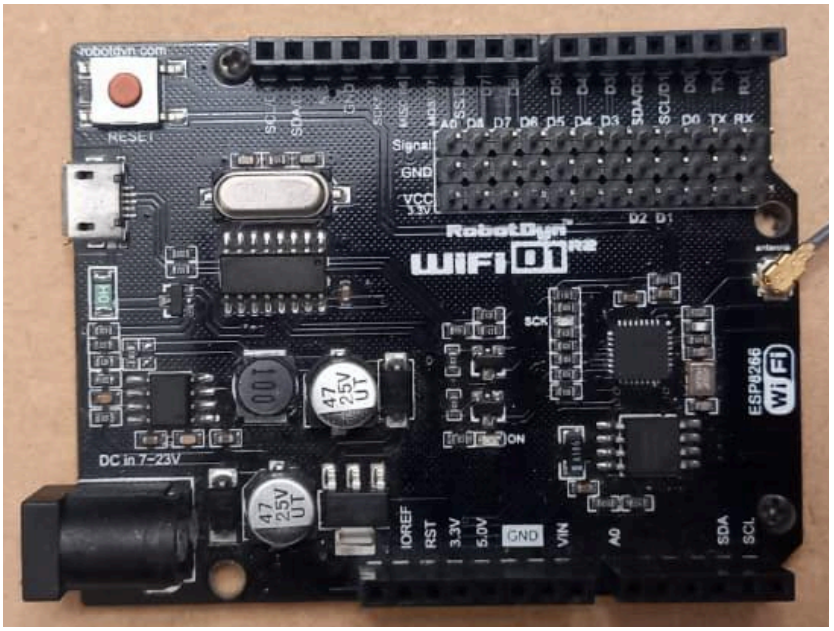

Figura 6: ESP8266 utilizado

- En la Figura 7, se presenta la interfaz web del sistema en funcionamiento, donde se ven todos los atributos descritos en la documentación de software.

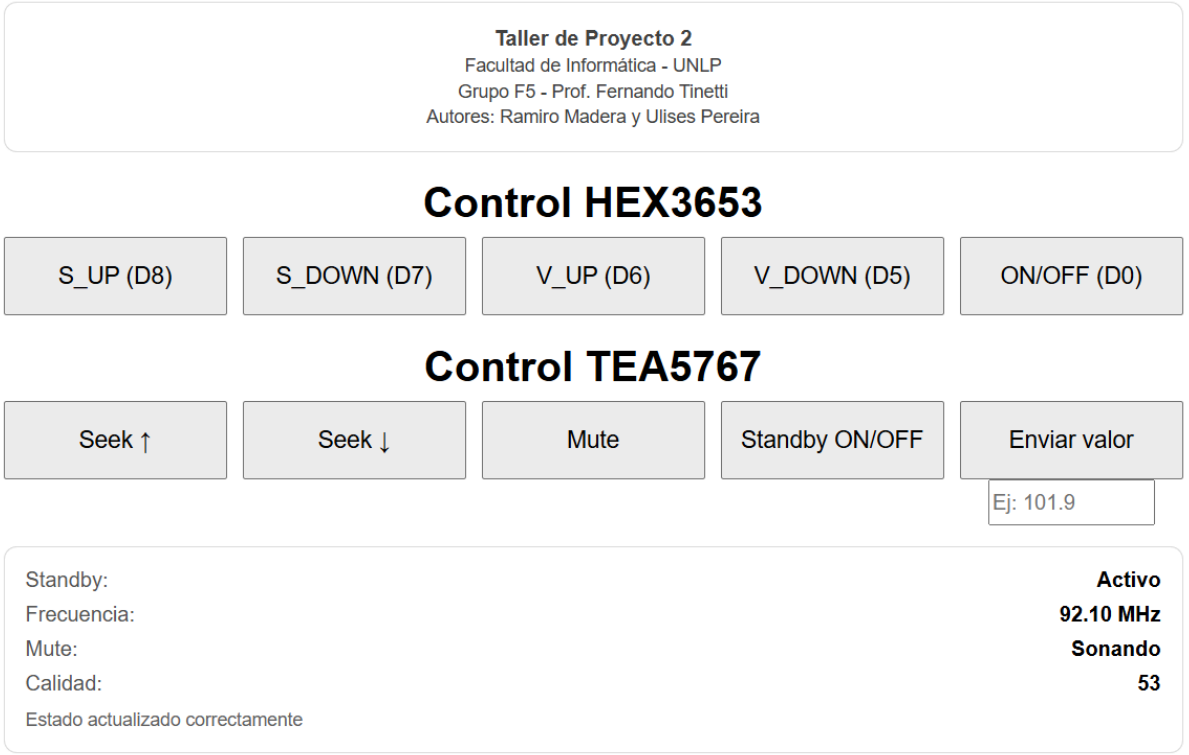

Figura 7: Interfaz Web del proyecto

- En el siguiente enlace se encuentra el video de corroboración de funcionamiento del proyecto, mostrando el cumplimiento de los requerimientos:
https://drive.google.com/file/d/1xh1CxHU-dW-fPM4u54sDsh_EraA9Rb04/view?usp=sharing
- En el siguiente enlace, se comparte la bitácora de trabajo del grupo, donde se han listado todos los avances, problemas, alternativas y soluciones que han sido elaboradas durante el desarrollo del proyecto:
https://docs.google.com/document/d/15Ns91AnPXT5aQHsSpQB_MCCK7EF7LCu1AMea9PLexQ4/edit?usp=sharing
- En la siguiente carpeta de Google Drive, se encuentran todos los archivos pertenecientes al proyecto:
https://drive.google.com/drive/folders/16PRvXzJ0SGg2CxCaohukSnp_7nwHrk-l?usp=sharing
- En el siguiente enlace, se encuentra la librería utilizada para manejar el receptor TEA5767:
https://github.com/vkumpan/TEA5767/tree/master
- Se listan a continuación las datasheets relevantes, o los documentos que fueron de utilidad en caso de no contar con hojas de datos, de los componentes utilizados en el sistema:
    - ESP8266: https://drive.google.com/file/d/1NUWg1mS0dj3hcl6syMN2dOc4YQZgRDiD/view?usp=sharing

- HEX3653: https://drive.google.com/file/d/1NUWg1mS0dj3hcl6syMN2dOc4YQZgRDiD/view?usp=sharing
- TEA5767:
- https://drive.google.com/file/d/1t9ltoCGDxBPpj8UjdXpl0N-Dkg6yMj-Q/view?usp=sharing
- KT0936:
- https://drive.google.com/file/d/1XtOQCYd4_eFO7wrfmS8OXZqlu7sYZWpX/view?usp=sharing

5.- Primera Extensión: Amplificación manejada por Hardware

5.1.- Descripción general

Al haber concluído satisfactoriamente con el control de ambos módulos, se propuso la opción de profundizar en alguno de los aspectos del sistema como parte de un nuevo proyecto de extensión, por lo que se optó por agregar una etapa de amplificación a la salida de audio, agregando la posibilidad de modular el volumen utilizando un potenciómetro.

Esta etapa requirió principalmente trabajo en el hardware; se agregó al sistema el uso del amplificador **PAM8403**, un potenciómetro de 10kΩ, un parlante de 8Ω y se pasó a utilizar una fuente de 5v debido a los nuevos requerimientos energéticos del sistema. También se optó por trabajar únicamente con el TEA5767, visto que en el desarrollo del proyecto original se pudo apreciar que el mismo provee de un reporte detallado de la señal y además carece de la posibilidad de control de volumen (no así como el HEX3653).

Los principales cambios en el apartado de software consistieron en remover las funciones que controlaban al HEX5767 en el proyecto anterior.

5.2.- Cambios en el Esquemático y Materiales

Se optó por utilizar el amplificador PAM8403 por que es un amplificador sencillo, versátil y de consumo relativamente acotado, además de que se cuentan con múltiples ejemplos de su uso en internet. Se trata de un amplificador de 6w de potencia y alimentación de 5v, lo cual se traduce en un consumo teórico de 1.2A, que representa un incremento significativo en relación al proyecto original, lo que motivó el reemplazo de la alimentación por pilas para pasar a la nueva fuente de 5v, a la cual se conectaron el PAM8403 y el ESP8266, mientras que se utilizó a éste último para alimentar al TEA5767 con 3.3V, del mismo modo que en el proyecto original.

En la Figura 8 se detalla el conexionado del nuevo sistema:

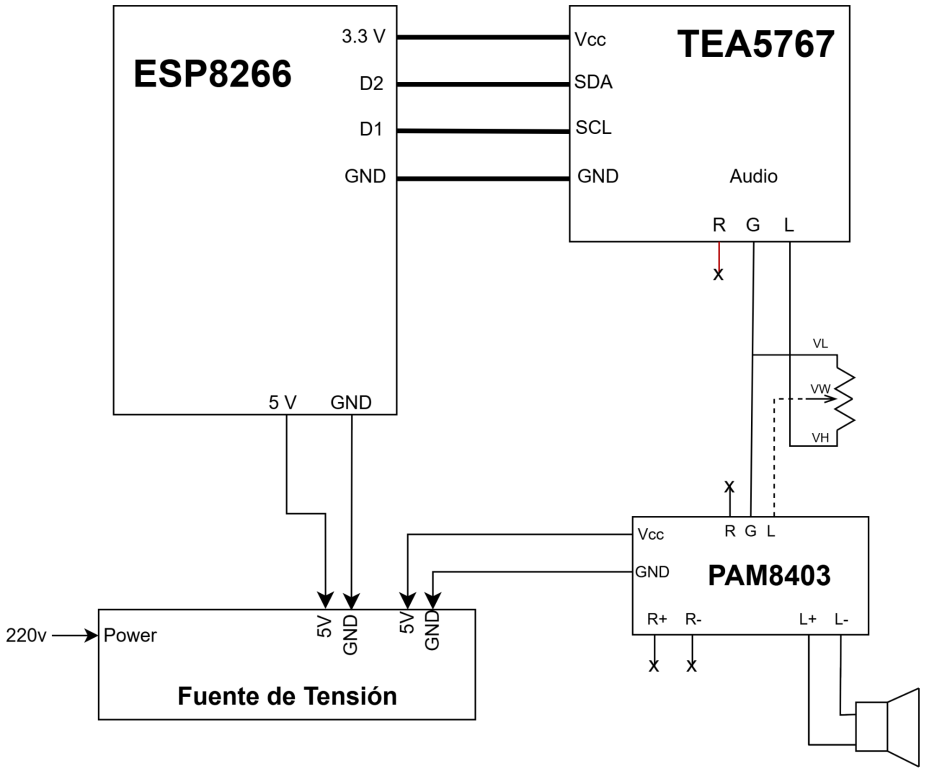

Figura 8: Diagrama de conexionado de la primera extensión.

5.3.- Cambios en el Software

Esta extensión del proyecto final está centrada principalmente en el agregado de una etapa de Hardware; en términos de Software, no se están incoporando nuevos requerimientos. Al contrario, se están eliminando los requerimientos relacionados al HEX 3653. Es por ello que la modificación del código consiste principalmente en el recorte de archivos y funciones. A continuación, se listan los cambios:

- Remoción de los archivos *HandlerHEX3653.cpp* y *HandlerHEX3653.h* del proyecto;
- En el archivo *.ino* que ejecuta en sí el sistema, se suprime el llamado al encabezado *HandlerHEX3653.h*, y se suprime la función *hexpins_init()* del código de inicialización;
- En *HTTPGeneral.cpp*, se suprime la inclusión de *HandlerHEX3653.h*, y el llamado a la función *hexHTTP_init()*;
- En *InterfazWeb.h*:
    - Se cambia el título de la página de “Control HEX3653 & TEA5767” a “Control TEA5767”;
    - Se elimina el apartado “Control HEX3653” y los 5 botones que contenía el mismo.

5.4.- Documentación Relacionada

- En la Figura 9, se presenta la interfaz web de la extensión en funcionamiento.

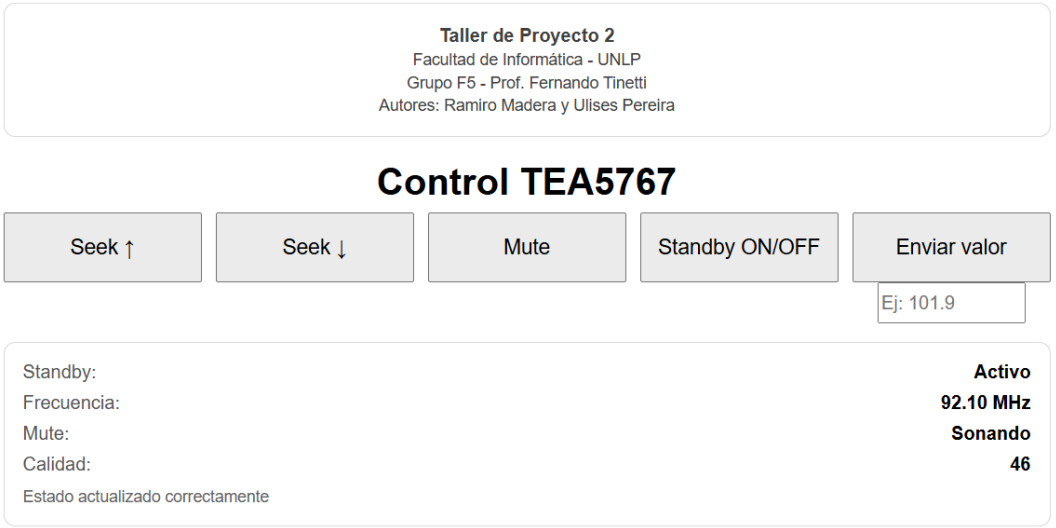

Figura 9: Interfaz Web de primera extensión

- En el siguiente enlace se encuentra el video de corroboración de funcionamiento de esta extensión, mostrando el cumplimiento de los requerimientos. Se aclara que en un momento se menciona erróneamente el “potenciómetro digital” queriendo hacer alusión al potenciómetro analógico:

https://drive.google.com/file/d/1g9jVrkjrj7f9PKy61nqAIblZwmW7h31-/view?usp=sharing

- En la siguiente carpeta de Google Drive, se encuentran todos los archivos pertenecientes a esta extensión:

https://drive.google.com/drive/folders/1qqROhA8rhjgLNVyqbuLsCrpsDCMH3yyk?usp=sharing

- Se adjunta el enlace a la hoja de datos del amplificador PAM8403 utilizado:

https://drive.google.com/file/d/1825n8ClmyO-xf_e51Quz0Oovvl5JGAhf/view?usp=sharing

6.- Segunda Extensión: Amplificación manejada por Software

6.1.- Descripción general

Una vez completa la extensión con el potenciómetro analógico agregado, se planteó una alternativa equivalente en cuanto a funcionalidad que unifique el control del receptor TEA5767 completamente en la interfaz web. Para ello, se reemplazó el potenciómetro analógico utilizado previamente por el potenciómetro digital **X9C103S**, que permite ajustar el volumen en pasos del 1% entre el mínimo y el máximo; de este modo, la interacción con el sistema será realizada únicamente desde el servidor web.

Esto implicó el reemplazo del potenciómetro analógico por conexiones al potenciómetro digital, junto con las conexiones requeridas para su alimentación. En cuanto a términos de Software, esta extensión consistió en agregar funciones y archivos al proyecto que permitan inicializar e interactuar con el nuevo dispositivo agregado al sistema.

6.2.- Cambios en el Esquemático y Materiales

En este caso el conexionado es similar a la primera extensión; se mantuvieron las conexiones de los 3 bornes del potenciómetro como en su contraparte analógica, pero el cambio a la versión digital requirió agregar la conexión de 3 puertos de GPIO del ESP8266 (D5, D6 y D7) para su control, además de la alimentación por medio de la fuente de 5v.

En la figura 10 se detalla el nuevo esquemático de conexiones:

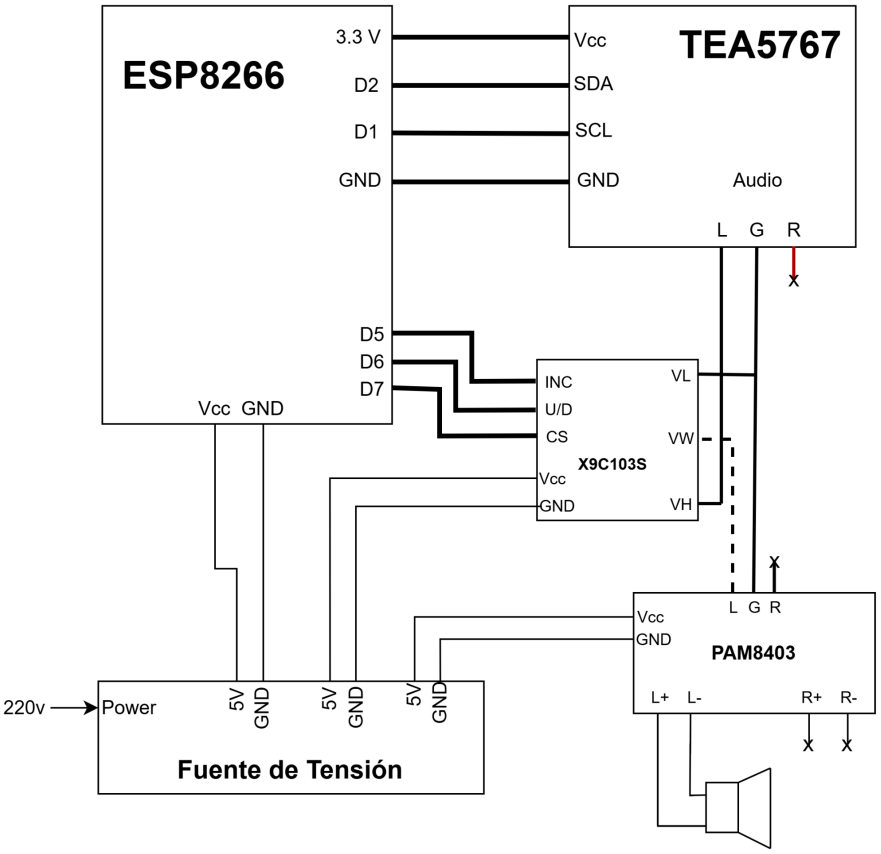

Figura 10: Diagrama de conexionado de la segunda extensión.

6.3.- Cambios en el Software

La extensión fue diseñada e implementada a raíz de la primera extensión hecha en el proyecto, agregando tanto en Hardware como en Software el uso de un potenciómetro digital que modifique la resistencia del amplificador. Es por ello que los cambios realizados se describirán en función al código de la primera extensión realizada.

A continuación, se listan las modificaciones:

- Se incluye la librería *X9C103S* rescatada de la plataforma Github, referenciada en la documentación relacionada, que controla el uso del dispositivo; cabe destacar que la misma fue agregada al proyecto y no fue listada en las librerías del Arduino instalado, por lo que no se requiere instalarla por fuera del proyecto. La misma define la clase **X9C103S** y cuenta con las siguientes estructuras y funciones:
    - El constructor X9C103S, tanto el vacío como el que configura los 3 pines *inc, ud* y *cs* que el dispositivo utiliza;
    - La función *initializePot()*, que debe ser llamada antes de utilizarlo para configurar los valores internos, particularmente el volumen inicial que comienza al máximo;
    - La función *setResistance(int)* que lleva el volumen al valor que se le mande por parámetro;
    - Las funciones *increaseResistance(int)* y *decreaseResistance(int)*, que incrementan o decrementan el valor del volumen en el valor indicado por parámetro. En estas funciones se realizaron correcciones: cuando las mismas llegaban a los extremos, modificaban el valor de la resistencia para que circule cíclicamente (es decir, de 1 baja a 100 y viceversa). Para que el código funcione bien, se necesita que los valores queden fijos en el extremo al que llegan;
    - La función *setToHighest()*, que lleva el volumen al máximo;
    - La función *setToLowest()*, que lleva el volumen al mínimo;
    - La función *getResistance()*, que devuelve el valor actual de la resistencia o volumen;
- Se incluyen los archivos *HandlerX9C103S.cpp* y *HandlerX9C103S.h*. En el archivo *.cpp* se encuentran las funciones que controlarán el uso del potenciómetro para el proyecto, y en el archivo *.h* se encuentran los encabezados que llamarán otros archivos. El desglose de los archivos es el siguiente:
    - En el encabezado, se declaran con macros los pines del ESP8266 asociados a los pines de comunicación que tiene el X9C103S;
    - La función *potfunc_init()*, que inicializa la clase controladora y establece el volumen en 50;
    - La función *potHTTP_init()*, que asocia la función *handler* con la ruta de cada botón de la interfaz web;
    - La función *updateVol()*, que es llamada luego de cada handler y tiene como objetivo enviar el nuevo valor de volumen a la interfaz web;
    - Las 7 funciones *handler,* asociadas a los 7 botones de control de volumen. todas tienen la misma estructura de llamar a una función de la clase potenciómetro y actualizar el volumen en el código HTML. Son las siguientes:
        - *handleInc1:* Incrementa el valor del volumen en 1;
        - *handleInc5:* Incrementa el valor del volumen en 5;
        - *handleInc10:* Incrementa el valor del volumen en 10;
        - *handleDec1:* Decrementa el valor del volumen en 1;

        - *handleDec5:* Decrementa el valor del volumen en 5;
        - *handleDec10:* Decrementa el valor del volumen en 10;
        - *handleSet(int):* Fija el valor del volumen al valor enviado por su parámetro;
- En el archivo *.ino* que ejecuta en sí el sistema, se agrega el llamado al encabezado *HandlerX9C103S*, y se suprime la función *potfunc_init()* del código de inicialización;
- En *HTTPGeneral.cpp*, se suprime la inclusión de *HandlerX9C103S.h*, y el llamado a la función *potHTTP_init()*;
- En *InterfazWeb.h*:
    - Se agregan los componentes *'pot-btn', 'pot-row'* y *'pot-input'* para armar el apartado de control de volumen;
    - En la función *enviarPulso*, se invoca a la carga de estado del potenciómetro en *cargarEstadoPOT()* para agregar los nuevos valores de volumen a la página;
    - Se agrega la función *enviarVolumen* para poder enviar un valor específico de volumen que se ingrese desde la interfaz web;
    - Se agrega la función *renderPot(state)*, que recibe el estado de volumen del potenciómetro y lo coloca en la página;
    - Se agrega la función *cargarEstadoPOT()*, la cual recibe el estado del volumen desde el backend del servidor y llama a la función *renderPot()* para mostrarlo en la interfaz;
    - En la función *renderTEA()*, se agrega lógica de control para ocultar el campo de volumen en la interfaz en caso de encontrarse el receptor muteado;
    - Al realizar la auto-actualización o refresco de la interfaz, se llama a la función *cargarEstadoPOT()* luego de cargar el estado del TEA;
    - Se agrega un apartado en el HTML para mostrar los 7 botones que modificarán el volumen, y se los asocia a la función del código a la que llamarán: los incrementos y decrementos enviarán un pulso a la ruta que corresponda, y el botón de volumen específico utilizará la función *enviarVolumen()*;
    - En el apartado inferior donde se devuelven los datos del TEA, se agrega una fila que muestre los datos del volumen, de ser correspondiente;

6.4.- Documentación Relacionada

- En la Figura 11, se presenta la interfaz web de la extensión en funcionamiento.

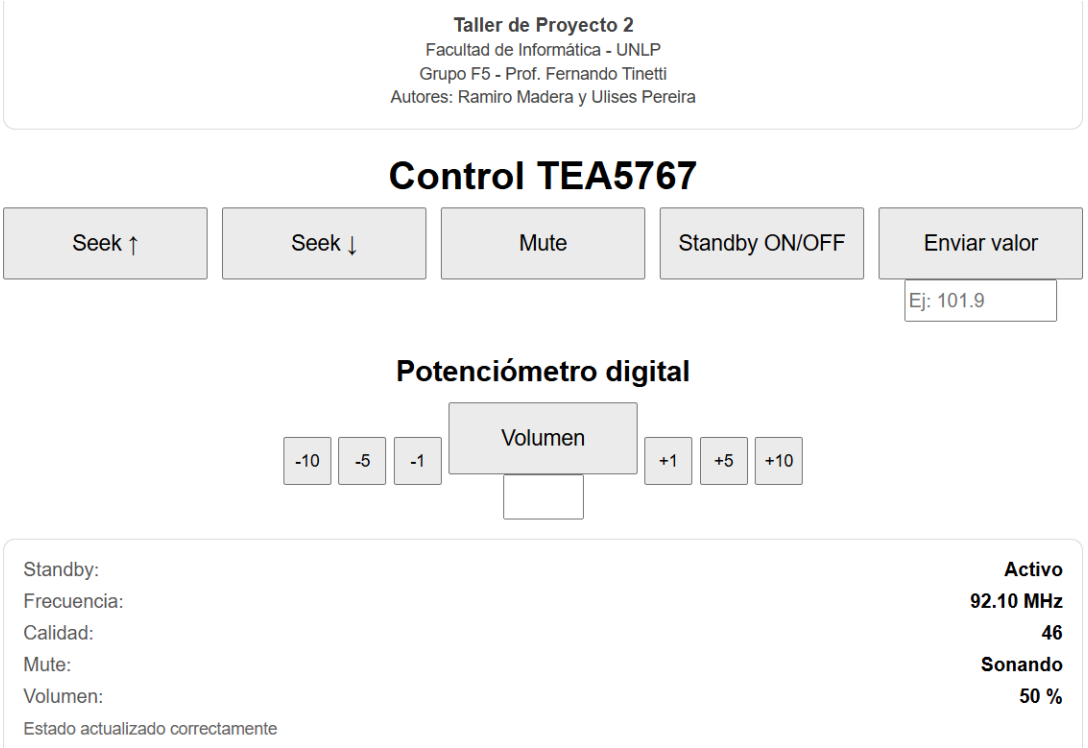

Figura 11: Interfaz Web de la segunda extensión

- En el siguiente enlace se encuentra el video de corroboración de funcionamiento de esta extensión, mostrando el cumplimiento de los requerimientos. Luego del grabado del mismo se realizaron 2 cambios en la estética de la interfaz: en primer lugar, se cambió el título de la página a “Control TEA5767”; y por otro lado, se reordenaron los botones de control de volumen para que aparezcan los que disminuyen el volumen a la izquierda y los que aumentan a la derecha, de modo que mientras menos es el incremento más cerca estén del centro.

https://drive.google.com/file/d/1JMiQW3l9mkdpLoay0C8mvjLdvJ3csJ3e/view?usp=sharing

- En la siguiente carpeta de Google Drive, se encuentran todos los archivos pertenecientes a esta extensión:

https://drive.google.com/drive/folders/1uBGSw6ytabVI-0HFGDe7gO6lAs_0oeEX?usp=sharing

- Se adjunta en el siguiente enlace la librería de un usuario utilizada para controlar el dispositivo X9C103S:

https://github.com/Hedrahexon/X9C103S

- Se adjunta el enlace a la hoja de datos del potenciómetro digital X9C103S utilizado:

https://drive.google.com/file/d/1825n8ClmyO-xf_e51Quz0Oovvl5JGAhf/view?usp=sharing

7.- Conclusiones Finales

A lo largo de los últimos meses de trabajo, se pudo construir exitosamente un proyecto controlador de receptores de frecuencias moduladas. Los principales objetivos propuestos en el plan de proyecto original fueron cumplidos con éxito, resultando así en un modelo completo y funcional para trabajar en primera instancia con los receptores HEX3653 y TEA5767 simultáneamente, y en segunda instancia con el receptor TEA5767 expandiendo sus capacidades en cuanto a la salida de audio.

Ambas extensiones realizadas al proyecto fueron completadas satisfactoriamente y aportaron mejoras a la calidad de la experiencia de usuario. El receptor elegido para esta parte del proyecto es el que más se beneficiaba de la mejora: a diferencia del HEX3653, el TEA5767 no cuenta con un control de volumen más allá del silenciado, por lo que el potenciómetro agrega funciones nuevas al sistema. Ambas variantes de la amplificación de volumen tienen su utilidad: mientras que el potenciómetro analógico permite pensar en un dispositivo integrado del cual pueda regular el volumen, el potenciómetro digital permite unificar el control del sistema en una única interfaz web.

En cuanto a posibles mejoras a realizar, el controlador del HEX3653 resulta de calidad satisfactoria; no obstante, la señal recibida por el TEA5767 cuenta con una interferencia significante, que se refleja en los valores de calidad devueltos por el dispositivo. El análisis de calidad de la señal y la implementación de filtros que clarifiquen la señal es una fuerte alternativa de extensión. Además, podría trabajarse en la mejora de la interfaz web: la misma presenta una contemplable latencia en su interacción, que debería ser analizada para poder plantear mejoras en la comunicación de los procesos. Una última opción de extensión sería la de explorar la posibilidad de un sistema que amplifique ambos receptores simultáneamente.

Apéndice A: Materiales y Presupuesto

En la tabla A1 se detallan los materiales utilizados para cada proyecto, junto con su precio, referencia y el costo total de cada uno de los proyectos terminados:

Tabla A1: Materiales utilizados para cada proyecto

Material	**Precio (USD)**	**Referencia**	**Proyecto Original**	**Extensión 1**	**Extensión 2**
ESP8266	4.15	**link**	✔	✔	✔
HEX3653	1.00	**link**	✔		
TEA5767	10.00	**link**	✔	✔	✔
Parlante USB	3.70	**link**	✔		
Cables para interconexionado	1.85	**link**	✔	✔	✔
2 pilas AA	1.00	**link**	✔		
Fuente 5v	4.00	**link**		✔	✔
PAM8403	1.00	**link**		✔	✔
Pot. analógico	0.75	**link**		✔	
X9C103S	4.80	**link**			✔
Parlante 4ohm	4.70	**link**		✔	✔
		Total:	21.70 USD	26.45 USD	30.50 USD